Visual Reverse Engineering of Binary and Data Files

Gregory Conti, Erik Dean, Matthew Sinda, and Benjamin Sangster

Department of Electrical Engineering and Computer Science
United States Military Academy
West Point, New York
{gregory.conti,erik.dean,matthew.sinda,
benjamin.sangster}@usma.edu

Abstract. The analysis of computer files poses a difficult problem for security researchers seeking to detect and analyze malicious content, software developers stress testing file formats for their products, and for other researchers seeking to understand the behavior and structure of undocumented file formats. Traditional tools, including hex editors, disassemblers and debuggers, while powerful, constrain analysis to primarily text based approaches. In this paper, we present design principles for file analysis which support meaningful investigation when there is little or no knowledge of the underlying file format, but are flexible enough to allow integration of additional semantic information, when available. We also present results from the implementation of a visual reverse engineering system based on our analysis. We validate the efficacy of both our analysis and our system with case studies depicting analysis use cases where a hex editor would be of limited value. Our results indicate that visual approaches help analysts rapidly identify files, analyze unfamiliar file structures, and gain insights that inform and complement the current suite of tools currently in use.

1 Introduction

Individual files are a fundamental component of today's computing paradigm as well as one of today's biggest threat vectors. With the advent of effective network security devices based upon firewalls, intrusion detection systems and similar security applications, attackers are moving away from network protocol attacks and toward attacking applications themselves. This transition is problematic because firewalls must pass some traffic in order to provide services to their users, particularly web and email. It is through these services that users send, receive, upload and download files, sometimes as email attachments, web downloads, or more worrisome, surreptitiously through encrypted channels such as HTTPS or SSH. The problem is worsened by the rapid evolution of file-based attacks that exploit vulnerabilities in parsing by applications and common software libraries, as well as by the attacker's use of packers which obfuscate the contents of files.

Legitimate files function as either stand alone executable programs or as data to be used by other applications, such as word processors, text editors or graphics programs.

J.R. Goodall, G. Conti, and K.-L. Ma (Eds.): VizSec 2008, LNCS 5210, pp. 1–17, 2008.
© Springer-Verlag Berlin Heidelberg 2008

Executable files are executed by the operating system, whereas, data files are loaded by applications. In both cases the operating system and application assume some knowledge of the file's underlying format and structure in order to operate on it successfully. In these legitimate cases, the user of the file only sees the interpreted version of the file as determined by the application or operating system. The underlying structure of the file is hidden. It is through this hidden nature that both attackers and legitimate programmers attempt to place security mechanisms. While file extensions and magic numbers serve the purposes of legitimate use, security analysts are forced to dig much deeper and face the frequent task of exploring the raw structure of files to detect modifications, determine file type, determine what a file does, determine authorship, create virus signatures, and detect code evolution, among other tasks. Further hampering analysis is that the file may be corrupted, obfuscated, or encrypted to slow analysis. In some cases, files contain code designed to crash analytic tools or function differently if used in a virtual machine. Sometimes files contain largely legitimate data with a small fraction containing a malicious aspect and others are dedicated malicious applications.

At their heart, files are just binary objects whose meaning is based on the applications or the operating systems that use them. However, in some cases little will be known about a given file. We therefore view the problem of reverse engineering binary files as having two levels, a context independent level where we assume the analyst has little or no knowledge about the structure and purpose of a given file, and analysis must occur initially in a context-independent manner. At the second level, the analyst knows some information, such as an expected file format, and can make informed assumptions as they analyze the file. The difference between these two levels is evident in the current suite of tools for reverse engineering files. At the context independent level, the analyst employs general purpose tools such as hex editors and byte frequency analysis to gain insight. At the semantic level, analyst tools are crafted specifically to a given file format or family of related formats, such as dissemblers and debuggers, leading to very precise insights.

Unfortunately, the vast majority of best of breed tools for reverse engineering of files are strictly text based and provide only a tiny viewport into the file under analysis. Visualization is underutilized in reverse engineering and bears great promise for assisting analysts in their work and augmenting their existing tool suite. In this paper we present an analysis of user tasks that is useful for the design of visualization systems, and employ this analysis to implement a system which combines proven text based techniques with innovative visualization approaches in order to augment the analyst and complement their tool suite for a number of essential tasks. We validate the efficacy of these contributions through several case studies. It is important to note that we are addressing the problem of reverse engineering of binary file formats across the entire range of possibilities and not focusing specifically on the special case of reverse engineering of executable files.

This paper is organized as follows. Section 2 places our research in the field of related work. Section 3 contains a user-level requirements analysis that we used to guide our development. Section 4 presents our system design and implementation. Sections 5 demonstrate the utility of our approach through case studies. Section 6 presents our conclusions and suggested directions for future work.